

FDA2 Near-Prime Loan Default — Project Report

Author: Mason Bennett **Kaggle username/display name:** Mason Bennett **Metric:** ROC-AUC **Final selected Kaggle upload file:** `predict_MLP60_top10.csv` **Final OOF AUC: 0.99237** | **Final leaderboard AUC: 0.9931**

Author's note. This project was completed under my direction for Financial Data Analytics II, a graduate course that explicitly encouraged the use of AI. I set the objective, steered the strategy across each stage, and reviewed and interpreted the workflow, results, and code; an AI assistant did the coding and much of the experimental brainstorming. The report is written in the first person for readability.

The final submission is a 60-seed bagged tanh-MLP trained on the top-10 features chosen by LightGBM importance, with feature selection refit inside every CV fold. This final refinement replaced the earlier L2-regularized logistic-regression stack because it lifted local OOF AUC from 0.98572 to 0.99237 and public leaderboard AUC from 0.9868 to 0.9931.

Read alongside `submission_mlp60_topk.py`, `make_mlp60_blends.py`, `submission_final.py`, `push_pca.py`, `push_pca_pseudo.py`, `push_final_bag.py`, `focus_top10_12.py`, `next_pseudo_top10.py`, `next_pseudo_iter3.py`, `next_pseudo_iter4.py`, `next_wider_top10.py`, `next_wider_pseudo.py`, `next_3way.py`, and `experiment_tracking.csv`. Every CV AUC below comes from the same `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` split, so OOF predictions are directly comparable.

1. Introduction

Binary classification of 30,000 near-prime consumer loans given 50 anonymised features `feat_00...feat_49` and 70,000 labelled training rows. Base rate $\approx$ 30% positive. The competition metric is ROC-AUC.

Going-in strategy. Establish every PDF-required baseline quickly, let EDA tell me where the signal lives, then push the advanced phase around whichever modelling lever the EDA pointed to. That turned out to be a six-stage journey: 1. **Pairwise-interaction feature engineering** broke the plateau from trees ($\sim$ 0.58) to MLPs ($\sim$ 0.95). 2. **PCA preprocessing** on the augmented matrix was the first major breakthrough ($\sim$ 0.974). 3. **Pseudo-labelling + bagged ensembling** of PCA $d=18$ + MLP runs reached OOF 0.97877 / LB 0.9799. 4. **Pivoting to top-10 features** (a hypothesis I tested after observing diminishing returns at top-18) shrank the augmented matrix from 689 to 238 columns; PCA at the new elbow $d=10$ lifted OOF to 0.982. 5. **L2 logistic-regression stacking** on logit-transformed OOFs from diverse model variants (chained pseudo-labelling using stack predictions as the source, a wider MLP backbone, 3-way interaction features) pushed OOF to **0.98572** / LB **0.9868**. 6. **Final pivot to direct top-K MLP bagging** used LightGBM importance as fold-refit feature selection, then a 60-seed tanh-MLP bag on only the stable top features. Top-12 produced LB **0.9922**; removing the unstable tail features and using top-10 lifted LB to **0.9931**.

Each stage's gain matched a specific design choice rather than blind capacity: the right features, then the right preprocessing, then the right ensembling.

2. Data and validation setup

- `train.csv`: 70,000 $\times$ 51 (50 features + `default`). `test.csv`: 30,000 $\times$ 51 (`id` + 50 features).

- All features are `float64`, target is `int64`. **No missing values.**
- Train positive rate: 0.3005. Features roughly z-scored (mean ≈ 0 , std ≈ 1 , range ± 5).
- **Univariate signal was zero.** Every feature had $\text{AUC} \leq 0.52$ and every squared feature the same. Every single-variable operation (sign, parity, threshold, sum, abs, squared, product) landed at ~ 0.50 . That shaped every downstream choice.
- **Feature distributions are exactly standard normal.** All 50 features have mean ≈ 0 , std ≈ 1 , skewness in $[-0.02, +0.02]$, and kurtosis in $[-0.04, +0.05]$. There are no skewed columns, no heavy tails, no positive-only columns. This rules out log / Box-Cox / quantile transforms as a path to additional signal — they have nothing to fix.
- **No train/test distribution drift.** Adversarial validation (LightGBM trained to distinguish train rows from test rows under 5-fold stratified CV) gave an OOF AUC of 0.498, indistinguishable from random. The two splits are from the same distribution; there is no temporal or batch-effect feature to exploit, despite the lender domain context implying loans across 2018-2023. The data is randomly shuffled, not ordered by origination date.

Preprocessing. - Tree models: no scaling, no imputation (data has no NaNs; trees are scale-invariant). - Linear models + MLPs: `SimpleImputer(median) → StandardScaler` inside a pipeline. - "Augmented" matrix (Section 4): the 50 raw features + pairwise products, sums, differences, absolute-differences of the top 18 features by LightGBM gain + row-wise summary statistics + 10 PCA components + 8 KMeans cluster-distance features. 689 columns total.

Validation. - 5-fold `StratifiedKFold(shuffle=True, random_state=42)`. Same fold assignment for every model. - OOF predictions for ranking and stacking. Test predictions fold-averaged. - For PCA+MLP: **PCA is fit inside each fold's training set only** — no leakage from held-out rows or pseudo-labelled test rows into the principal components.

3. Required baseline model results

Model	Preprocessing	Key parameters	CV AUC	Std
LogisticRegression	median-impute + standard-scale	$C=1.0$, solver=lbfgs	0.50073	0.00851
RandomForest	none	n_estimators=120, min_samples_leaf=20, max_features=sqrt	0.54437	0.00627
HistGradientBoosting	none	lr=0.05, max_iter=800, max_leaf_nodes=63	0.56938	0.00545
LightGBM (baseline)	none	lr=0.03, num_leaves=63, min_child_samples=50	0.58141	0.00581

What these taught me. - **Logistic regression = 0.500.** Random. Perfectly consistent with all univariate AUCs being 0.50. No linear combination of features has linear signal. - **Random Forest** at `max_features='sqrt'` only sees ~ 7 of the 50 features per split — too few to find interaction signal. - **HistGB / LightGBM baseline** both sit at 0.57-0.58 because they can find a little interaction via tree depth but not nearly enough.

A LightGBM probe showed AUC climbed monotonically with `num_leaves` ($31 \rightarrow 63 \rightarrow 127 \rightarrow 255 \rightarrow 511$) suggesting *more tree capacity*. But empirically the signal turned out to be too subtle for trees alone; the right move was neural nets.

4. Advanced experiments

4.1 Pairwise-interaction feature engineering (the first breakthrough)

A fast probe on an 85/15 split showed:

Feature matrix (LGBM num_leaves=255)	AUC
raw features	0.584
+ pairwise products of top 15 features	0.681 (+0.098)
+ quad (products + sums + differences + abs-diffs) of top 15	0.724 (+0.141)
raw, num_leaves=2047 (pure capacity)	0.584 (no gain)

Tree capacity alone was NOT the answer. Only explicit pairwise transformations of top features broke the plateau. The 689-col augmented matrix is the product of this probe.

4.2 Gradient-boosted trees on augmented features

- **LightGBM** (num_leaves=255, min_child_samples=15, feature_fraction=0.4, lr=0.03), 5-fold CV: **0.72274** OOF
- **CatBoost** (depth=7, lr=0.06, Bernoulli subsample), 5-fold CV: **0.74197** OOF

Significant jump over raw baselines (0.58), but trees on the augmented matrix plateau.

4.3 sklearn and PyTorch MLPs on augmented features

- sklearn `MLPClassifier(256,128,64)` single seed: 0.842
- sklearn `MLPClassifier(512,256,128)` 3-seed bag: 0.915
- PyTorch MLP (1024, 512, 256, 128) with GELU + BatchNorm + Dropout(0.3) + AdamW + cosine LR, 3 seeds: **0.947**
- Same arch, 7 seeds: **0.95029**

MLPs leapt past trees by +0.20 AUC on the same features. The target is locally smooth once the right pairwise transformations are present — trees carve axis-aligned rectangles; MLPs fit a smooth function directly.

4.4 PCA preprocessing (the SECOND breakthrough)

After hitting a plateau at OOF 0.95 with progressively richer feature engineering, I formed a working hypothesis: the 689-column augmented matrix had too many noisy directions for the MLP to fit cleanly, and *reducing* dimensions before the MLP would filter the noise. PCA was the obvious lever to test. I tested PCA on multiple input matrices:

Probe	AUC (single-split MLP)
aug689, no PCA	0.934
aug689, PCA d=18	0.959
aug689, PCA d=22	0.961
aug689, PCA d=50	0.931
aug689, PCA d=100	0.847
aug689, PCA d=200	0.793

A sharp sweet spot appears around $d=18-22$. Interpretation: only ~ 20 principal directions carry signal; the other 669 of 689 dimensions are noise the MLP overfits. The Week 12 workbook's "Dimensionality Reduction for Supervised Learning" (Section 12.6.3) describes exactly this mechanic — PCA as noise filtering before a supervised model.

Scaled up to 5-fold CV with 3-seed bagging:

Config	OOF AUC
PCA $d=18$ + MLP, 3 seeds	0.97393 ← best PCA config without pseudo-labels
PCA $d=20$ + MLP, 3 seeds	0.97151
PCA $d=22$ + MLP, 3 seeds	0.97078
PCA $d=20$ + concat raw 50 + MLP	0.879 (hurts – raw features re-inject noise PCA removed)

$d=18$ is the winner. +0.02 OOF over the same MLP without PCA.

4.5 Pseudo-labelling

Took very-high-confidence test rows ($\text{prob} < 0.02$ or > 0.98 from `pca_d18_s3`) and appended them to training, then re-ran PCA+MLP. The validation fold stayed on original training data so OOF AUC is still honest.

Iteration	Pseudo rows added	OOF AUC
Iter 1 (threshold 0.02/0.98)	11,646 (10,796 neg + 850 pos)	0.97736 (+0.003)
Iter 2 (threshold 0.01/0.99, using iter 1 preds)	10,951	0.97785 (+0.0005)

Diminishing returns, consistent with how much extra signal the test set rows can add after the first pass.

4.6 Seed bag expansion

PCA $d=18$ with pseudo-labels, bagged across **7 seeds** (42-48): OOF **0.97873**. Small additional lift over 3 seeds. Combined with the iter1 and iter2 runs into a greedy-mean ensemble at OOF **0.97877** — the first leaderboard checkpoint at **LB 0.9799** and the anchor for everything that followed.

4.7 Pivot to top-10 features (third breakthrough)

After locking in LB 0.9799 with the top-18 pipeline I started questioning the choice of K . The features beyond rank ~ 10 in the LightGBM gain ranking had noticeably lower importance scores than the top 10, and their pairwise interactions with the top features were adding $C(K, 2) = K(K-1)/2$ columns each — most of them likely carrying more noise than signal. I hypothesized that **reducing K from 18 to 10** would shrink the augmented matrix substantially while preserving the actually-informative pairwise interactions, and that the $d=K$ elbow rule (Section 4.4) would let the MLP work in a tighter, denser principal-component space. The augmented matrix shrank from 689 to 238 columns: 50 raw + 9 row-wise summary statistics + 4 pairwise operations $\times C(10,2)=45$ pairs = 180 pairs. Per the $d=K$ elbow rule, the PCA target shifted from $d=18$ to $d=10$ in lockstep.

Config	5-fold + 3-seed OOF AUC	Δ vs. prior best
top-18 PCA d=18 + 3 pseudo iters + 7-seed bag (prior best)	0.97877	(LB 0.9799)
top-10 PCA d=10 + MLP, 3-seed CV	0.98176	+0.003 OOF (single config beat the full prior ensemble)
top-12 PCA d=12 + MLP, 3-seed CV	0.97996	+0.0012

Same fold split, same MLP architecture, same training procedure — just a smaller, denser augmented matrix.

The pivot decision is the genuinely interesting bit. An earlier single-fold probe on top-10 PCA d=10 had given AUC 0.97488 — *worse* than the same probe on top-18 (0.97415). Treating that as a kill signal would have closed the door. But this was a single-split, single-seed probe, while the pipeline that actually tracks the leaderboard is 5-fold CV with 3-seed bagging. With the right pipeline the ranking flipped and the gain was substantial. **Trust the evaluation pipeline you'll actually use, not a cheaper proxy.**

4.8 Pseudo-labelling on top-10 + iter3 with stronger pseudo source

Three new pseudo-labelling iterations on the top-10 augmented matrix, each with the same recipe as iter1/iter2 but using progressively stronger source predictions:

Iter	Source predictions	Pseudo rows added (neg + pos)	OOF AUC
iter1	top-10 base (OOF 0.98176)	12,395 (11,401 + 994)	0.98335
iter2	iter1 (OOF 0.98335)	11,159 (10,721 + 438)	0.98327
iter1 + iter2 50/50 mean	—	—	0.98370 → LB 0.9846

Iter2 alone saturated against iter1 — its high-confidence test rows overlap heavily with iter1's. So I tried a different angle: **use a stronger source** for the next iteration's pseudo-labels.

The stronger source was the L2 logistic-regression stack of all the existing models (Section 4.9), with OOF 0.98398. Pseudo-labels from a stronger source are cleaner pseudo-labels.

Iter	Source predictions	Pseudo rows added	OOF AUC
iter3	LR-stack (OOF 0.98398)	20,288 (16,034 + 4,254)	0.98466
iter4	iter3 (OOF 0.98466)	14,261	0.98370 (saturated)

iter3 outperformed iter1, iter2, and iter4. Source quality > iteration count when the recipe is otherwise the same.

4.9 Logistic-regression stacking on OOFs

Greedy mean blending stopped finding gains around OOF 0.98370. With ~10 model OOFs to combine — many of them highly correlated (Spearman $\rho \geq 0.994$) — a meta-model that learns weighted combinations should beat a simple mean. Every meta-model was trained with its own 5-fold CV on the OOFs (no leakage; the OOFs are already cross-validated, so the meta-CV is honest).

Meta-model	Inputs	OOF AUC
Greedy mean	top model OOFs	0.98370
LightGBM (small: 100 trees, 8 leaves)	7 model OOFs	0.98365
Logistic regression (probs, C=1.0)	7 model OOFs	0.98398 → LB 0.9851
Logistic regression (probs, C=1.0)	8 OOFs (with iter3 added)	0.98508 → LB 0.9858
Logistic regression on logit(OOF), standardized, C=10	8 OOFs (with iter3)	0.98529 → LB 0.9863
Logistic regression on logit, standardized, C=10	11 OOFs (with wider, 3-way)	0.98572 → LB 0.9868 (final)

Three observations: 1. **LR > LGBM as a stacker** when the inputs are highly correlated. LGBM tries to find non-linear splits in a near-linear OOF space and overfits the noise that distinguishes the models. 2. **Logit-transforming the inputs before LR helped consistently** by ≈ 0.0002 OOF. Probabilities are bounded to [0,1] and squashed at extremes; logits are unbounded and approximately linear in the underlying score, so LR fits them more naturally. 3. **C tuning saturated fast** — $C \in \{0.1, 1.0, 10.0, 100.0\}$ all landed within 0.00005 OOF of each other once the input transformation was right.

4.10 Diversifying the stack: wider MLP and 3-way interactions

Once L2 logistic-regression stacking was the meta-model, additional lift required *diverse* models, not bigger versions of the same recipe. Two independent additions:

New stack member	Single-model OOF	Stack OOF after add
Wider MLP (1024, 512, 256) on top-10 PCA d=10, 5-seed bag	0.98229	LR_logit_std stack → 0.98558 (+0.0003)
Wider MLP + iter3-source pseudo-labelling (3-seed)	0.98372	(added in same step as above)
3-way pairwise products of top-10 (358-col aug, PCA d=10, 1 seed)	0.97946	LR_logit_std stack → 0.98572 (+0.00012, FINAL)

Both individual models were *worse* than iter3 alone on their own OOF, but each contributed real signal to the stack — the wider MLP via architectural diversity, the 3-way products via a different feature basis. The 3-way interaction lift is small (+0.00012 OOF, +0.0005 LB) but consistent with the stacker's behavior: it can find a marginal residual signal even from a marginal model member, as long as that member is genuinely different.

4.11 Final top-K tanh-MLP bag

After the stacked model reached LB 0.9868, the final improvement came from revisiting a much simpler recipe: use LightGBM only as a feature selector, then train a bagged neural net directly on the selected raw features. The important detail is leakage control: **LightGBM feature ranking is refit inside each CV fold**, then the MLP trains only on that fold's selected features. This keeps the feature-selection step inside the validation loop.

The recipe in `submission_mlp60_topk.py`:

- Selector: `LGBMClassifier(n_estimators=200, num_leaves=63, learning_rate=0.05, min_child_samples=20, random_state=42)`.
- Model: `MLPClassifier(hidden_layer_sizes=(128, 64), activation="tanh", alpha=1e-3, batch_size=256, max_iter=300, early_stopping=True)`.
- Bagging: seeds `0..59` inside each fold; predictions averaged across the 60 bag members, then averaged across folds for the test set.

- CV: the same 5-fold `StratifiedKFold(shuffle=True, random_state=42)` used throughout the project.

Config	OOF AUC	Public LB	Notes
MLP60, LightGBM top-12 per fold	0.99125	0.9922	Strong intermediate recipe; included unstable 11th/12th features
MLP60, LightGBM top-10 per fold	0.99237	0.9931	Final selected submission; all five folds selected the same stable 10-feature core
95/5 probability blend of top10/top12	0.99235	not selected	Slightly lower OOF than pure top10
90/10 probability blend of top10/top12	0.99233	not selected	Also lower OOF than pure top10
MLP60, LightGBM top-11 per fold	pruned at 2 folds	not submitted	First two folds averaged 0.99229 vs top10's same-fold 0.99260
MLP60, LightGBM top-9 per fold	pruned at 1 fold	not submitted	Fold 1 AUC collapsed to 0.79579 after dropping <code>feat_02</code>

The top-10 fold AUCs were `0.99214, 0.99306, 0.99223, 0.99211, 0.99254`. The selected feature sets differed only in order; every fold used the same stable feature core:

`feat_00, feat_01, feat_02, feat_04, feat_09, feat_14, feat_32, feat_34, feat_36, feat_38`.

This result changed my interpretation of the earlier top-K work. The PCA+interaction pipeline showed that ranks beyond the top 10 were often noise-amplifying. The final raw-feature MLP confirmed the same pattern more sharply: top-12 was excellent, but top-10 was better because the 11th and 12th selected features varied by fold (`feat_15`, `feat_16`, `feat_18`, `feat_22`, `feat_30`, `feat_33`, `feat_44`) and acted like fold-specific noise.

Follow-up top-K checks reinforced the same boundary. Top-9 was pruned after fold 1 because removing `feat_02` caused fold AUC to collapse to 0.79579. Top-11 was much more plausible, but its first two folds averaged 0.99229, below top10's same-fold average of 0.99260, so it was pruned under the pre-declared completed-fold rule. These checks made the final top-10 choice more defensible: K=10 is not just a convenient round number; it is the point where all critical features are included and the unstable tail is still excluded.

4.12 Negative results worth noting

Top-18 era: - **Rich matrix with ALL pairwise products** (1811 features): single-fold AUC dropped to 0.697. Too much noise drowned the signal. Pure dimensionality is bad without reduction. - **Plain torch MLP on raw 50 features**: 0.580 single-split. Need the pairwise features. - **KNN (any K, any feature matrix)**: max 0.61. Signal isn't in distances. - **Logistic regression on polynomial-degree-2 features (1325 cols)**: 0.509 regardless of C. Target is non-linear in feature space. - **LightGBM on polynomial-degree-2 features**: 0.570. Tree models can't navigate the wide feature matrix. - **PLS supervised dimensionality reduction**: 0.50-0.59 at small d. Couldn't find target-correlated low-dim subspaces. - **LDA projection**: 0.50-0.52 (one component for binary). Same reason. - **PCA d<15 on aug689**: 0.50-0.72 — below d=18 is a cliff. - **PCA d=17 with pseudo labels**: 0.83690. A specific principal component at index ~18 carries critical signal; d=17 loses it entirely (sharp cliff). - **PCA d=20 + concatenated raw features**: 0.88-0.90. Adding raw back in re-introduces the noise PCA had removed.

Top-10 era (overnight session): - **KernelPCA with RBF kernel** at $\gamma=0.005$ fold-1 OOF was 0.887; at $\gamma=0.05$ fold-1 OOF was 0.543 (worse than random because the kernel was too peaked). Linear PCA dominates because features are already exactly $N(0,1)$ — the linear structure is the right structure. - **Univariate-AUC ranking** for top-10 features picked five different features than the LightGBM-gain top-10. Trained MLP gave OOF 0.5617 — essentially random. Univariate AUC

is the wrong feature ranker for this problem because the signal is in interactions, not marginal effects. - **Different LGBM seeds for feature ranking** killed within seconds of launch: seed 99 produced an *identical* top-10 to seed 42, despite a different `train_test_split` random seed. The LGBM gain ranking is highly stable, so this approach yielded zero diversity. - **Pseudo-labelling iter4 chained from iter3** OOF 0.98370 (worse than iter3's 0.98466). Pseudo-labelling saturates after iter3 when the source predictions are already strong. - **Feature transforms (log, Box-Cox, quantile)** on the top-10 features were not viable: feature distributions are exactly $N(0,1)$ with skew $|\leq 0.02|$ and kurtosis $|\leq 0.05|$, so power transforms have nothing to fix. - **Adversarial train/test discriminator** OOF AUC was 0.498 — no temporal drift, no `id`-based exploit available. The data is randomly shuffled despite the lender domain context implying loans across 2018-2023. - **Wider MLP single config** (1024, 512, 256) on top-10 PCA $d=10$ gave OOF 0.98229 — only +0.0005 over the standard (512, 256, 128) at 0.98176. Width alone doesn't help on this PCA-projected, low-dimensional input. The wider model still earned its place in the stack via architectural diversity, not single-model performance. - **3-way interactions ALONE** were worse than 2-way: top-10 PCA $d=10$ with 120 triple products added (358 cols) gave OOF 0.97946 single-seed vs. 0.98076 single-seed for 2-way. The additional features introduced noise the MLP overfit. The 3-way model still earned a place in the stack because the LR meta-model could exploit the *direction* of its residual error even though the absolute OOF was worse.

5. Ensembling

Two ensembling phases produced distinct lifts:

Phase 1 — greedy mean blend (top-18 era). Greedy forward selection on OOF predictions with mean-blend and rank-blend strategies. The PCA+MLP family had narrow, highly correlated prediction distributions (Spearman $\rho \geq 0.994$ across models), so blends were driven primarily by the strongest few PCA+MLP models. Final greedy-mean ensemble of `{pca_pl_d18_s7, pca_pl_iter2, pca_pl_iter1}` at OOF **0.97877** → LB **0.9799**.

Phase 2 — L2 logistic-regression stacking on logit-transformed OOFs (top-10 era). Once the model count grew (iter3 with stacked source, iter4, wider MLP, wider+pseudo, 3-way interactions) and individual gains saturated, greedy mean stopped finding lift. A logistic-regression meta-model on logit-transformed and standardized OOFs lifted the ensemble from greedy 0.98370 to LR-stacked **0.98572** → LB **0.9868**. LightGBM as meta-model regressed (overfit OOF noise); LR with logit-transformed inputs was the right call. See Section 4.9 for full meta-model ablations.

Phase 3 — probability blends around MLP60. After the MLP60 top-10 result, I tested conservative probability blends of the stronger top-10 model with the weaker but slightly different top-12 model. The two submission vectors were highly correlated (Pearson ≈ 0.9989), so the blends were intentionally small: 95/5, 90/10, 85/15, 80/20, 75/25. Local OOF declined monotonically as top-12 weight increased; pure top-10 remained the chosen final submission. This was a useful negative result because it showed the top-12 tail features were not adding helpful ranking diversity.

6. Final model and submission

- **Chosen submission:** 60-seed bagged tanh-MLP on the LightGBM-selected top-10 features, with LightGBM feature ranking refit per CV fold. Final code: `submission_mlp60_topk.py`.
- **Final OOF AUC: 0.99237** (fold AUCs: 0.99214, 0.99306, 0.99223, 0.99211, 0.99254).
- **Final leaderboard AUC: 0.9931.**
- **Competition file:** `predict_MLP60_top10.csv` — 30,000 rows, columns `id` and `default_probability`, header matches `sample_submission.csv`, no NaNs, probabilities in $[0, 1]$. This is the best confirmed public leaderboard submission in the project folder.

- **Prior final retained for audit trail:** the earlier L2 logistic-regression stack (`predict.csv`) remains in the folder and is described in Sections 4.8-4.10. It reached OOF 0.98572 / LB 0.9868 before the MLP60 top-K refinement superseded it.
- **OOF→LB calibration.** The earlier OOF-to-LB gap was reproducibly $\approx +0.001$ across six consecutive submissions. The final MLP60 runs showed the same general behavior but at a higher level: top-12 OOF 0.99125 → LB 0.9922, top-10 OOF 0.99237 → LB 0.9931. This consistency made the top-10 submission worth prioritizing over lower-OOF blends.

7. Experiment tracking table

Final MLP60 submissions and checks:

Model / experiment	CV AUC	LB	Output file	Notes
MLP60 top-10 LightGBM features	0.99237	0.9931	<code>predict_MLP60_top10.csv</code>	Final selected submission
95/5 top10/top12 probability blend	0.99235	–	<code>predict_blend_top10_95_top12_05.csv</code>	Validated, but local OOF below pure top-10
90/10 top10/top12 probability blend	0.99233	–	<code>predict_blend_top10_90_top12_10.csv</code>	Validated, but lower OOF
MLP60 top-11 LightGBM features	pruned at 2 folds	–	–	First two folds trailed top10 same-fold mean
MLP60 top-12 LightGBM features	0.99125	0.9922	<code>predict_MLP60_top12_report_match.csv</code>	Strong report-match recipe; superseded by top-10
MLP60 top-9 LightGBM features	pruned at 1 fold	–	–	Dropping <code>feat_02</code> collapsed fold-1 AUC

Earlier project-era experiments are sorted by 5-fold OOF AUC below (see `experiment_tracking.csv` for full table). Submissions confirmed on the public leaderboard are noted in the LB column.

Rank	Model / experiment	CV AUC	Std	LB	Notes
1	LR_logit_std stack with 3-way (FINAL)	0.98572	—	0.9868	Final submission. L2 LR on logit-transformed OOFs of 11 models
2	LR_logit_std stack with wider+iter3src pseudo	0.98558	—	—	Penultimate stack; superseded by 3-way addition
3	LR_logit stack (8 OOFs incl iter3)	0.98529	—	0.9863	First logit-transformed stack; OOF→LB gap +0.0010
4	LR_prob stack with iter3 (8 OOFs)	0.98508	—	0.9858	First time iter3 was added to a stack
5	pl_top10_iter3_lsrc (3 seed bag)	0.98466	0.00072	—	Pseudo-label iter3 with LR-stack as source
6	LR_prob stack (7 OOFs, no iter3)	0.98398	—	0.9851	First LR stacking submission
7	iter1+iter2 50/50 mean (top-10)	0.98370	—	0.9846	First top-10 ensemble submission
8	pl_top10_iter4 (3 seed bag)	0.98370	0.00099	—	Saturated past iter3
9	pl_top10_wide_iter3src (3 seed bag)	0.98372	0.00060	—	Wider MLP backbone + iter3 pseudo source
10	pl_top10_iter1 (3 seed bag)	0.98335	0.00066	—	First top-10 pseudo iteration
11	pl_top10_iter2 (3 seed bag)	0.98327	—	—	Saturated past iter1
12	top10_pca10_wide_s5	0.98229	—	—	Wider MLP (1024,512,256), 5-seed, no pseudo
13	top10_pca10_pseudo_blend_lr_logit	0.98209	—	—	Earlier intermediate
14	top10_pca10_s3 (3 seed bag)	0.98176	0.00067	—	Single-config top-10 baseline (the pivot win)
15	pl_top12_iter2 (3 seed bag)	0.98195	0.00077	—	
16	pl_top12_iter1 (3 seed bag)	0.98163	0.00064	—	
17	top12_pca12_s3 (3 seed bag)	0.97996	—	—	top-12 baseline
18	top10_3way_pca10_s1	0.97946	—	—	3-way interaction features, single seed
19	greedy_mean_ensemble (top-18 era)	0.97877	—	0.9799	Original final, became baseline for overnight push
20	pca_pl_d18_s7	0.97873	0.00110	—	Top-18 7-seed bag with pseudo
21	pca_pl_iter2	0.97785	0.00081	—	Top-18
22	pca_pl_iter1	0.97736	0.00054	—	Top-18

Rank	Model / experiment	CV AUC	Std	LB	Notes
23	pca_p1_d19_s3	0.9758 3	0.0013 3	—	Top-18, d=19
24	pca_d18_s3	0.9739 3	0.0013 8	—	Best PURE PCA d=18 (no pseudo)
25	pca_d20_s3	0.9715 1	0.0012 1	—	Top-18 era
26	pca_d22_s3	0.9707 8	0.0019 9	—	Top-18 era
27	torch_big_7s	0.9502 9	0.0025 8	—	No PCA, 7-seed deep MLP
28	torch_mlp_big (3 seeds)	0.9471 8	0.0043 1	0.956 8	Earlier submission
29	torch_huge_3s	0.9410 7	0.0028 3	—	Even larger torch MLP
30	sklearn_mlp_big_seeds3	0.9152 0	0.0042 9	0.931 2	First ~0.90 submission
31	cat_deep_aug	0.7419 7	0.0045 4	—	CatBoost on augmented
32	lgbm_deep_aug	0.7227 4	0.0045 6	—	LightGBM on augmented
33	lgbm_base	0.5814 1	0.0058 1	—	LightGBM baseline (required)
34	histgb	0.5693 8	0.0054 5	—	HistGB baseline (required)
35	top10_pca10_uvauc_s3	0.5616 6	0.0056 0	—	Univariate-AUC ranking; negative result
36	random_forest	0.5443 7	0.0062 7	—	RF baseline (required)
37	logreg	0.5007 3	0.0085 1	—	LR baseline (required; random)

8. Reflection

Biggest surprise: PCA preprocessing was the decisive move. I spent the first chunk of the project adding features (pairwise products, sums, differences, PCA components appended). That got us to 0.95 OOF. Then a pass through Week 12 of the workbook ("Dimensionality Reduction for Supervised Learning", Section 12.6.3) reframed the problem for me: *reducing* features via PCA before the MLP — rather than adding more — was the next thing to try. It promptly added +0.02 AUC. The final d=18 sweet spot is sharp: d=17 catastrophically loses a critical component (AUC 0.84), d=22 drops off gradually (0.97), d=50 hurts (0.93), d=100+ drowns the signal entirely (0.85 and falling).

Second surprise: the single most impactful EDA step was computing the univariate AUC of each feature in the first five minutes of the project. All 50 features had $AUC \leq 0.52$. That ruled out linear models, told me the signal was in interactions, and drove every subsequent modelling choice. Fast EDA is worth its weight in compute.

Compute-constrained experiment design (a methodology that emerged from hardware limits).

A real bottleneck for this project was hardware. Each fit took 150–300 seconds on CPU, with occasional outliers exceeding 20 minutes from overnight system-suspend artifacts. At 5-fold × 3-seed × multiple configurations per experiment, total wall-clock per experiment was 1–3 hours. Spending three hours on an experiment that turned out to be a dud was simply unaffordable — so I ended up adopting **early-kill heuristics** for experiments rather than letting them all run to completion.

The pattern that emerged: 1. **Define a kill criterion before launching** — a minimum AUC for the first fold or first seed below which the experiment is discontinued. For example, when launching a 3-way-interaction experiment I declared in advance that I would kill it if fold-1 AUC fell below 0.97. 2. **Watch first-fold / first-seed results** rather than waiting for the full 5-fold OOF, and reallocate compute to the next experiment if the early signal is weak. 3. **Watch for non-AUC early signals** — when launching an experiment that re-ranked features with a different LGBM seed, the first print statement showed seed 99 produced an identical top-10 to seed 42. Killed the process within seconds because there was no diversity to be gained, regardless of where the AUC would have landed.

Concrete examples from this project: - **push_topK** (a 5-K-value sweep over top-K augmented matrices) was killed mid-run when individual fits started exceeding 1200s due to system-suspend artifacts. We pivoted to `focus_top10_12.py`, which ran only the K=10 and K=12 configurations that actually mattered for the strategy. - **KernelPCA RBF**: at gamma=0.005, fold-1 OOF was 0.887 vs. linear PCA's 0.98 on the same data. Killed before the gamma=0.5 sweep ran — there was no plausible reading where wider RBF kernels would suddenly beat linear PCA after gamma=0.005 and gamma=0.05 both massively underperformed. - **Diverse-LGBM-seed ranking**: the first print after launch showed seed 99's top-10 features were identical to seed 42's. Killed immediately — no diversity to gain. - **3-way interactions on top-10**: pre-declared kill threshold of fold-1 AUC < 0.97. The experiment cleared the threshold (fold-1 was 0.98009) and was allowed to complete, where it produced a small but real stack lift of +0.00012 OOF.

This is a real ML engineering practice. The formal versions are **successive halving** and **Hyperband** in the hyperparameter optimization literature, both of which allocate compute disproportionately to promising configurations and prune underperforming ones early. Top performers with better hardware may not have needed this discipline; the hardware constraint forced me to develop it from first principles. The reframing matters: limited compute doesn't just mean "fewer experiments" — it means "more disciplined experiment design," which is closer to how production ML is actually done than infinite-resource Kaggle workflows.

Third surprise: the OOF-to-LB gap was nearly constant. Across six consecutive submissions, the leaderboard AUC was always within 0.001 of the local 5-fold OOF AUC (mean +0.00099, std 0.00018). This is a property of the *evaluation procedure*, not the *models*: a stratified 5-fold split on 70k rows estimates LB to four decimal places consistently. Once I noticed this, I could predict each LB before submitting and decide which improvements were worth a submission slot. Several modelling tweaks I would have otherwise submitted speculatively were skipped because the OOF said the LB delta would be negligible. This kind of empirical calibration is mundane but operationally valuable.

Things that worked: - 5-fold stratified CV with a fixed seed, shared across every model — OOF predictions directly comparable and stackable. - Pairwise interaction features of the top-K features by LightGBM gain (brought LGBM from 0.58 to 0.72; structurally identical at top-18 and top-10). - PyTorch MLP with GELU + BatchNorm + Dropout + cosine-annealed AdamW on the augmented matrix (0.95 on aug689; 0.98+ on top-10 aug238 + PCA d=10). - **PCA d=K preprocessing** before the MLP — d=18 with top-18, d=10 with top-10. The "elbow at d=K" rule holds across feature-set sizes. - **Pivot from top-18 to top-10 features.** I hypothesized that the lower-importance features in positions 11–18 were adding more pairwise-interaction noise than signal, and that the d=K elbow would let the MLP work in a tighter principal-component space. Single-config OOF lifted by +0.003. - **Final raw top-10 MLP60 bag.** The strongest model did not need the engineered/PCA matrix at all: LightGBM-selected raw top-10 features plus a heavily bagged tanh-MLP reached OOF 0.99237 / LB 0.9931. The lesson is that once the correct tiny feature subset was isolated, ensembling a smooth low-dimensional learner was more powerful than adding more engineered inputs. - **Pseudo-labelling with strong source predictions.** iter1 and iter2 use the previous model's predictions; iter3 broke saturation

by sourcing pseudo-labels from a logistic-regression stack (a stronger model than any individual MLP). +0.001 OOF over chained iter1→iter2. - **L2 logistic-regression on logit-transformed and standardized OOFs** as the meta-model. Beat greedy-mean and LightGBM-stacking variants because the OOF inputs were highly correlated ($\rho \geq 0.994$); a linear meta-model with a sensible input transform was the right tool. - **Architectural diversity in the stack** (wider MLP, 3-way interaction features). Each individual model was worse than iter3 alone, but each contributed unique residual signal that the linear stacker exploited. - **Compute-disciplined experiment design** with pre-declared kill criteria (see the methodology subsection above).

What I would try next with more time: 1. **Autoencoder bottleneck** as a non-linear replacement for PCA. A $238 \rightarrow 10 \rightarrow 238$ autoencoder on the top-10 augmented matrix might find non-linear low-dim structure linear PCA misses. With GPU compute this becomes practical at the scale we'd need. 2. **TabNet or FT-Transformer** with its own attention-based feature selection, as an architecturally independent stack member. CPU training was prohibitive for transformer-based tabular models on this dataset; with GPU access this would have been an obvious fifth or sixth member of the stack. 3. **Snapshot ensembling**: train one MLP with a cyclic learning rate, save model weights at each cycle's minimum, ensemble those snapshots. Cheaper than multiple full retrains. 4. **Higher-count bagging or parallelized CPU training** for the final raw top-10 tanh-MLP. The 60-seed bag already worked well; a larger bag may reduce variance slightly, especially if run in parallel across CPU cores or on a GPU implementation. 5. **Look for a structural exploit I missed**. Before the final top-K MLP60 pivot, the OOF curve had flattened around 0.985 despite trying multiple feature transformations, model architectures, and ensembling strategies. That kind of flat ceiling suggests there is a specific feature combination, transformation, or test-set property that maps cleanly to the label which my augmentation + PCA + MLP recipe doesn't capture. Adversarial validation ruled out distribution drift, so it is not a temporal/batch effect. With more time I would re-examine raw feature pairs and triplets directly for exact-or-near-exact predictive structure rather than relying on the MLP to discover it. 6. **Final retraining on all 70k train rows + high-confidence pseudo rows**, with no fold hold-out, for the submission predictions only (CV still used to validate the recipe). Typically adds +0.001-0.003 LB.

Outside resources used

- **Project instructions PDF** — set the workflow shape, defined required baselines.
- **Week 9-12 PDFs (FDA2 course material)** — specifically Week 12 Section 12.6.3 "Dimensionality Reduction for Supervised Learning" explicitly describes PCA-as-preprocessing for MLPs; this was the framing that made the PCA breakthrough click. Section 12.7 on "Combining PCA and Clustering" is adjacent.
- **scikit-learn documentation** — confirmed `PCA(n_components=d)` with `fit_transform` pattern for safe per-fold use; confirmed `MLPClassifier(early_stopping=True)` works as expected.
- **PyTorch documentation** — `BatchNorm1d`, `GELU`, `AdamW`, `CosineAnnealingLR` for the MLP regularization and training.
- **LightGBM / CatBoost docs** — "more leaves + less regularization" for deep-interaction signals.
- **Empirical OOF-to-LB calibration**. After my second submission I noticed the gap was approximately +0.001 OOF→LB; over the next four submissions this held to within ± 0.0002 . Treating "predicted LB $\approx$ OOF + 0.001" as a working hypothesis let me prioritize submission slots and reject several speculative tweaks that would have produced negligible LB lift.
- **AI assistant workflow** — planning, code drafting, debugging, iterating on probes, interpreting results, and final synthesis. AI use is explicitly encouraged by the project policy (Section 6.2 of the project instructions). I reviewed and understood the submitted workflow, results, and code.